Brewing SaaS - Architecture & Implementation Plan (AI-first)

Status: v0.3 (living document)
Primary goal: ship a BrewersFriend-style product with **native mobile apps** (marketing + brew-day reliability) and a **desktop-first web app** (workhorse), built with an **AI-first workflow** (Cursor + GPT-5.2).
Guiding principle: keep the backend *boring* (monolith, predictable patterns), make offline reliable without early sync complexity, and keep decisions easy to review and evolve.

---

0. Current "implemented architecture decisions" (high signal)

This section describes decisions that are now **implemented in the repo**.

0.1 Supported locales are single-source-of-truth

- **Canonical locale ownership**: `packages/i18n` (`@brewery/i18n`)
- Implemented exports:
  - `locales` (readonly tuple)
  - `SupportedLocale`
  - `defaultLocale`
  - `isLocale(value)`
  - `getSharedMessages(locale)` (full message tree)

Web integrates via a thin re-export: - `apps/web/src/i18n/routing.ts` re-exports from `@brewery/i18n` so web can keep its existing structure while avoiding drift.

0.2 Cross-platform routing boundary (route IDs + typed params)

We do **not** try to share Next.js routes or file-based routing across web/native. Instead we share a **route manifest** that allows: - shared screens to navigate without importing Next.js or React Navigation modules - push notifications / deep links to target stable route IDs later - explicit policy for "ported vs not ported" flows

Implemented package: - `packages/navigation` (`@brewery/navigation`) - `RouteId`, `RouteParamsById`, `RouteRef` - `routeToPath(RouteRef)` produces a **non-locale** web path (e.g. `/inventory`, `/recipes/:id/water/mash`) - `getRouteAvailability(id, platform)` returns: - `available` (web) - `blocked` (native default) - `whitelisted_web_fallback` (native, for safe webview fallback candidates) - `WEBVIEW_WHITELIST_ROUTE_IDS` currently starts with: `inventory`

0.3 "Block-first + whitelist webview fallback" policy (native)

Policy direction (agreed): - If a user hits a not-yet-ported route on native, show **"Not available on mobile yet"** first. - Some routes may be **webview whitelisted** for later fallback (read-only / safe surfaces). - Inventory is the first example whitelist candidate; later candidates may include MPR console.

0.4 Universal i18n React hook boundary (`useT` + `rich`)

To share screens across web and native while keeping message syntax consistent, shared code must not import `next-intl` directly.

Implemented package: - `packages/i18n-react` (`@brewery/i18n-react`) - Universal runtime: - `LocaleProvider({ locale, messages })` - `useT(namespace)` returning `{ t(key, values), rich(key, values) }` - Native-ready formatting uses ICU via `intl-messageformat`, fed by `getSharedMessages(locale)` - Web adapter entrypoint (optional): - `@brewery/i18n-react/next-intl` provides a `useT(namespace)` implemented via `next-intl` (thin wrapper)

0.5 Web adapter scaffolding (Next.js)

Web keeps Next App Router + `next-intl` locale-prefixed URLs.

Implemented file: - `apps/web/src/navigation/appRouter.ts` - `useAppRouter()` implements `AppRouter` over `next-intl` navigation + locale prefixing - It uses `routeToPath()` from `@brewery/navigation` and prefixes `/${locale}`.

0.6 Cross-platform API client (fetch boundary + auth)

Implemented package: - `packages/api-client` (`@brewery/api-client`) - Uses a minimal cross-platform fetch contract (injectable `fetch`) and avoids DOM-only typing in its public API. - **Auth direction (current)**: - Web: cookie sessions (`sid`) via `cookieAuth()` - Native: **bearer-only** via `bearerTokenAuth(getToken)` - Node (if used): treat as **bearer-only** - **Webview caveat**: opening a web route inside a native webview is not automatically authenticated by the native bearer token. If we want "already logged in" webviews, we must implement an explicit bridge (cookie/session handoff or token → webview session mechanism).

0.7 Webview auth bridge (bearer → cookie session handoff)

To support the "block-first + whitelist web fallback" direction without weakening the core auth split (native bearer-only, web cookie-session), the API implements a **system-browser-first** bridge that:

- starts from a **native bearer session**
- mints a short-lived, single-use exchange code
- then exchanges it for a normal web cookie session (`sid`) and redirects to a safe in-app path

Implemented pieces:

- DB model: `services/api/prisma/schema.prisma` → `WebviewExchangeCode` (`webview_exchange_codes` table)
  - stores `code_hash` (never stores raw code), `session_id`, `user_id`, `expires_at`, `used_at`
- Routes: `services/api/src/routes/auth.ts`
  - `POST /auth/webview-exchange` (bearer-only)
    - body `{ next: "/en/<path>" }`
    - response `{ ok, code, expiresAt, bridgeUrl }` where `bridgeUrl` is under `/api/auth/webview-bridge?...` (nginx rewrite)
  - `GET /auth/webview-bridge?code=...&next=...`
    - validates `next` is a safe locale-prefixed relative path (`/en...` or `/it...`)
    - validates and consumes the code (single-use, 60s TTL)
    - creates a normal cookie session and `302` redirects to `next`

This is the mechanism that enables "Continue on web" from native for whitelisted routes while being **already logged in** in the system browser.

0.8 Database routing foundation (pgpool-II + sync replication + auto-degrade)

To enable "single URL" scaling (Magento-cloud-like) while keeping **auth/session correctness**, the repo implements a production-like local stack with:

- **Postgres primary + hot standby** (streaming replication)
- **Replication slot + WAL archive** so a standby can catch up without re-seeding after outages
- **pgpool-II** as a **single DB entrypoint** (`DATABASE_URL`)
- **Synchronous replication when healthy** (`remote_apply`) to keep replica reads consistent
- **Auto-degrade** to primary-only when the replica is unhealthy/lagging (preserves availability and correctness)

Implemented pieces:

- Compose wiring: `docker-compose.yml`
  - `postgres`, `postgres-replica`, `pgpool`, `db-guard`
- Postgres durability:
  - archive volume: `wal_archive` mounted at `/wal-archive`
  - slot: `replica1`
- Guard:
  - `infra/db-guard/db-guard.sh` toggles `synchronous_standby_names` and pgpool standby attach/detach
- Prisma "safe lane" for migrations:
  - `services/api/prisma/schema.prisma` uses `directUrl = env("DATABASE_URL_DIRECT")`

Docs:

- `docs/Posgres-master-slave-replicas-architechture.md`
- `docs/PGPOOL-VERIFICATION.md`

---

1. Product goals and non-goals

Goals

- **Native apps are mandatory** (iOS/Android app stores). Native is not a "PWA replacement"; it is the differentiator:
  - brew-day must work with poor/no connectivity
  - offline-first logging
  - store presence for marketing
- **Desktop web is the workhorse** for serious workflows (recipe building, analysis, administration).
- **AI-first development**:
  - reduce cognitive load and context switching
  - prefer mainstream conventions and type safety
  - code is shipped with tests (unit + e2e) from day 1
- **Keep billing simple**:
  - start with Stripe (few tiers)
  - keep Paddle as a later option (billing adapter boundary)

Non-goals (for v0)

- Level 3 local-first sync (multi-device conflict-free merges) unless demanded by customers.
- Complex GraphQL schema governance (start with boring REST + projection).
- Microservices, event sourcing, CQRS, custom infrastructure.

---

2. Big picture architecture

Chosen stack

- **Web app (desktop + mobile web):** Next.js + React + TypeScript
- **Native apps (iOS/Android):** React Native + Expo + TypeScript
- **Backend monolith API:** Node.js + Fastify + TypeScript
- **ORM / schema management:** Prisma
- **Central database:** PostgreSQL
- **Offline database (device):** SQLite (native apps)
- **Reverse proxy ("doorman"):** Nginx
- **Local dev:** Docker Compose (routing parity: Nginx container included)
- **CI/CD:** GitHub + GitHub Actions

- **Deployment:** Bare metal VPS initially; Postgres on same VPS initially; dedicated DB VPS later if needed

Monorepo sharing strategy (web + native)

- Shared packages that cross the web/native boundary are treated as **buildable packages**:
  - runtime: `dist/**/*.js`
  - types: `dist/**/*.d.ts`
- Shared UI uses Tamagui, with a dedicated `@brewery/ui` package and platform-specific config entrypoints (web vs native) to avoid importing web-only dependencies in native.
- **Strict placement rule**: if code might be reused in native, it lives under `packages/**` first.

Cross-platform boundaries (routing + i18n)

Boundary rule: shared screens must not import platform frameworks If code is intended to be shared between web and native (screens/flows/components), it must **not import**: - `next/*` modules - `next-intl/*` modules - React Navigation modules - Expo Router modules

Instead it depends on small shared interfaces: - routing: `@brewery/navigation` - i18n: `@brewery/i18n` + `@brewery/i18n-react`

Implemented boundary modules (source of truth)

- **Locales + messages**: `packages/i18n` (`@brewery/i18n`)
- **Universal translation hook**: `packages/i18n-react` (`@brewery/i18n-react`)
- **Universal route IDs + policy**: `packages/navigation` (`@brewery/navigation`)
- **Web adapter**: `apps/web/src/navigation/appRouter.ts` (`useAppRouter()`)

Route policy: avoid accidental "not ported" drift We treat porting as an explicit capability decision: - every route has a stable `RouteId` - native can mark a route as: - **blocked** (default) - **available** (ported) - **whitelisted_web_fallback** (safe webview candidate later)

This keeps the "not ported yet" state deliberate, and avoids silent divergence.

Why these choices (summary)

- **TypeScript everywhere** reduces context switching and increases AI effectiveness (types catch mistakes).
- **Fastify** keeps backend simple and fast; encourages explicit composition.
- **Prisma** provides schema-driven workflow and strong developer ergonomics.
- **PostgreSQL** is a strong general-purpose DB, great for evolving SaaS needs.
- **SQLite on device** provides true offline persistence (brew-day reliability).

- **Nginx** is familiar and great for TLS, routing, and deployment predictability.
- **Route manifest + adapters** keeps Next App Router stable while adding native without forcing a router migration.
- **Universal i18n hook** preserves your ICU message investment and supports `.rich()` patterns cross-platform.

UI information architecture (IA) refactor policy (web + mobile)

Principle: separate "input pages" from "results pages"

- **Input pages** are for changing data and running calculations (mash inputs, sparge inputs).
- **Results pages** are only introduced when results become large/complex enough that:
  - users need a stable "report" view (shareable/exportable),
  - the UI needs multiple subpanels, or
  - we want a read-only audit surface that avoids recomputation.

Criteria for adding `water/results`

We should add a dedicated results page (e.g. `/recipes/:id/water/results`) only when at least one is true: - **The results surface is too big** to comfortably keep on the input pages without cognitive overload. - **We need a stable snapshot view** driven only by the last saved calculation(s), with no double calculation. - **We want an export/share workflow** (print/PDF/email) that benefits from a dedicated page.

Until then, keep only small **read-only summaries** and deep links from input pages/hub.

---

3. Request flow and responsibilities (LEMP analogy)

Nginx is the doorman. Next.js is the web app. Fastify is the backend.

Runtime responsibilities

- **Nginx**
  - TLS termination (or delegated to a managed edge later)
  - route `/api/*` to Fastify
  - route `/` to Next.js
  - rate limits, request size caps, basic hardening
- **Next.js (web)**

  - web routing, SSR/CSR rendering, web UI
  - calls backend via `/api`
- **Fastify (API)**
  - authentication/authorization (ACL)
  - business rules
  - persistence (Postgres via Prisma)
  - Stripe webhooks + entitlement updates
  - background jobs (later)
- **Expo (mobile)**
  - offline-first UI
  - local SQLite persistence
  - sync queue pushing events to backend when online

---

4. Tenancy model and ACL (Magento-like)

Tenancy

- **Account/team-owned** model from day 1.
- Users belong to one or more accounts (clubs/breweries) with roles.

Roles

User-facing role names: - `owner` - `brewery-admin` - `member` - `viewer`

Implementation note: - for database identifiers, use `brewery_admin` (underscore), while the UI can show `brewery-admin` (dash).

Authorization approach

- **Application-level authorization** (Fastify middleware + service-layer checks) is the baseline.
- **Row Level Security (RLS)** may be added later for defense-in-depth, but is not required for v0.

Data ownership rule

All domain tables include `account_id`. Every read/write is scoped by: - authenticated `user_id` - chosen `active_account_id` - membership + role checks

Support tooling requirement: "login as brewery" (impersonation)

We want a support-only capability to reproduce user-reported bugs: - "Login as brewery/account" (and/or "login as user within account") - must be **audited** (who impersonated whom, when, why) - restricted to specific privileged support roles / feature flags Implementation is deferred; this is a design constraint to keep in mind.

---

5. Offline strategy: reliable Level 2 now, defer Level 3

Offline requirements

- Brew-day workflows must **never lose data**.
- Measurements and notes are stored locally immediately.

Level 2 offline (chosen)

- **Write locally first** (SQLite) with `sync_state = pending`.
- Sync loop retries until server acknowledges events.
- Server endpoints are **idempotent** by client-generated `event_id` (UUID).

Append-only events: the key simplification

Instead of "editing the same record", brew-day data is modeled as immutable events: - gravity/temp/pH measurements - notes - timer start/stop - corrections (as new events referencing the superseded event)

This makes sync "add missing events" and avoids most conflicts.

Level 3 (deferred)

Multi-device offline edits with conflict resolution is not planned unless demanded. Design still keeps the door open via: - append-only event model - versioning on mutable entities (recipes) - explicit conflict policies if introduced later

---

6. API design: boring REST + projection

Base rule

Start with straightforward REST endpoints: - `/accounts`, `/members`, `/recipes`, `/batches`, `/events`

Projection (GraphQL-like benefits without GraphQL)

- `?include=` for related data
- `?fields=` for selective field projection

Sync endpoints

- `POST /sync/push` - send local events batch (idempotent)
- `GET /sync/pull?since=` - fetch server-side changes since cursor/token

Public recipe submission (web + API)

We want a public-facing path to submit recipes (for community/public pages): - Web form should be protected (captcha and rate limiting). - The same capability must be possible via API (for automation/AI agents), protected by API keys/tokens and rate limits. Implementation is deferred; this is a design constraint.

---

7. Data model: overview and domain constraints

Core entities (high level)

- Account, AccountMember (roles)
- User, Device
- Recipe
- Batch
- BrewEvent (append-only)

Water profiles are part of the recipe (day-1 rule)

A water profile is not "external metadata"; it is a core part of the recipe and must obey: - exactly **one water profile per recipe** (1:1) - it cannot be linked to multiple recipes - it can be **duplicated** to another recipe (copy creates a new profile record) - water profile editing is limited to water correction: - only water amounts and salts/acids adjustments - do not edit grist/malt bill or hop schedule from water profile screens - we maintain **a single source of truth** for values: - the water profile is the authoritative data - recipe views can display values copied/derived from the profile, but edits must happen in one place only

We will need database entities for: - salts - acids - water profiles and water additions

Single source of truth principle (global rule)

We must avoid storing the same conceptual data in two editable places. Reason: offline sync and recalculation become unreliable when updates do not propagate deterministically.

---

8. Brew-day steps, reminders, and safety

We want support for custom, user-defined brew-day steps (including safety steps): - examples: "close LPG valve", "sanitise pump", "open chiller bypass", etc. - steps can be defined at: - brewery profile level (defaults) - specific batch/brew session level (overrides/additions) - steps can trigger notifications/reminders (future)

This is a key product differentiator and a strong reason to prefer native apps.

---

9. Testing strategy and testability conventions

Principles

- Keep tests "boring" and useful.
- Put correctness into **core domain library** (pure functions).

Stack

- **Unit tests:** Vitest (packages/core, services/api)
- **Integration tests:** API + DB (compose-based test DB or testcontainers later)
- **E2E tests:** Playwright (web)
- **Mobile:** start with unit + a small number of integration tests; add heavier mobile e2e after flows stabilize

E2E selector convention: `data-testid` (balanced rule)

To keep Playwright reliable and reduce brittle selectors: - add `data-testid` to workflow-critical elements: - primary buttons (save/create/delete/sync) - key form fields - key navigation items - modals/toasts that confirm important actions - do not add `data-testid` to purely presentational components

Prefer accessibility selectors (`getByRole`, `getByLabel`) when stable, and use `data-testid` as the stable fallback for custom/duplicated/dynamic UI.

---

10. Deployment approach (VPS + Docker)

Local dev

- Docker Compose with:
  - `nginx`
  - `web` (Next.js dev)
  - `api` (Fastify dev)
  - `postgres`
- Expo mobile dev runs on host or a container depending on preference.

Production

- Bare metal VPS:
  - Nginx on host or container
  - Docker Compose running web/api
  - Postgres initially on same VPS

- Later:
  - move Postgres to dedicated VPS or managed provider
  - add PgBouncer when connection pressure appears
  - add read replicas when read-heavy scaling requires it

---

11. Cursor-ready implementation plan (phases)

Phase 0 - Repository and standards

- Create monorepo structure (`apps/`, `services/`, `packages/`, `infra/`, `docs/`)
- Use `docs/architechture-Rev02.md` as the source-of-truth for Cursor + humans
- Native-ready packages (Metro-safe boundaries):
  - Shared packages imported by native apps ship `dist/**/*.js` + `dist/**/*.d.ts`
  - Do not export raw TS at the runtime boundary for native-consumed packages
  - Strict placement rule: if code might be reused in native, it lives under `packages/**` first
- Cross-platform boundaries (implemented early to reduce rework):
  - locales/messages: `@brewery/i18n`
  - route manifest + policy: `@brewery/navigation`
  - universal translation hook: `@brewery/i18n-react`
  - web adapter: `apps/web/src/navigation/appRouter.ts`
- Shared UI direction:
  - Keep Tamagui tokens/config/components in `packages/ui`
  - Split Tamagui config into web vs native entrypoints to avoid importing web-only drivers in native (e.g. CSS animations)
- Shared TS configs + linting
- CI skeleton (typecheck + unit)

Phase 1 - Backend foundation (Fastify + Prisma)

- Bootstrap Fastify server with:
  - request context (`userId`, `activeAccountId`, role)
  - auth middleware
  - error handling + structured logs
- Prisma schema + migrations
- Seed scripts (dev)

Phase 2 - Accounts + ACL

- CRUD:
  - create account

    - invite/join members
    - role management (owner/brewery-admin/member/viewer)
- Enforce ACL centrally in service layer.

Phase 3 - Domain core + first features

- `packages/core`:
    - brewing calculations (pure functions)
    - validation and unit conversions
    - strong unit tests
- API endpoints for recipes and batches (scoped to account).

Phase 4 - Append-only events + sync

- Data model for `BrewEvent`
- `/sync/push`:
    - validate membership
    - idempotent insert by `event_id`
    - return ack list
- `/sync/pull`:
    - return changes since cursor (timestamp or server token)

Phase 5 - Web app (Next.js)

- Auth + account switcher
- Desktop-first UI for recipes/batches
- Playwright e2e baseline
- Mobile web responsive support + "Open in app" prompts

Phase 6 - Mobile app (Expo)

- Local SQLite schema mirrors event model
- Brew-day mode:
    - local event capture
    - sync queue + retries
- Deep links:
    - open specific batch from web links

Phase 7 - Billing (Stripe) + entitlements

- Stripe checkout flow
- Webhook ingestion (store StripeEvent idempotently)
- Entitlement updates
- Feature gating and plan limits (API enforced; UI mirrored)
- Keep provider adapter boundary so Paddle can replace later.

Phase 8 - Deployment hardening

- Nginx config (routing parity)
- Docker production compose
- secrets management
- monitoring and backups
- optional PgBouncer when scaling app instances

---

12. Living constraints (the "always apply" rules)

- Keep backend monolithic and boring.
- REST-first. Add include/fields before considering GraphQL.
- Offline reliability is a product guarantee (native apps).
- Append-only events for brew-day logs.
- Account/team tenancy + role-based ACL enforced server-side.
- Support-only "login as brewery/account" capability (audited).
- Public recipe submission (web + API), protected by captcha/keys and rate limits.
- Water profile is part of recipe (1:1) with single source of truth; salts/acids DB.
- Avoid duplicated editable data in multiple places (single source of truth).
- Brew-day custom steps + safety reminders are first-class (future notifications).
- Tests ship with features (unit + Playwright baseline).
- Add `data-testid` to workflow-critical UI elements (balanced rule).

Appendix A - Initial Prisma schema (v0.2)

(Intentionally unchanged from Rev01; update when schema stabilizes.)

Appendix B - Local development Docker Compose (outline)

(Intentionally unchanged from Rev01; runtime stack changes require explicit coordination.)

Appendix C - CI outline (GitHub Actions)

Minimum pipeline stages: 1. install + cache 2. typecheck (TS) 3. unit tests (Vitest) 4. build (web + api) 5. e2e (Playwright) against a compose stack (optional early, recommended soon)